

RSA Encryption

1. Choose two large prime numbers p and q such that $p \neq q$, randomly and independently of each other. Set $N = pq$.
2. Pick integer e coprime with $\phi(N) = (p - 1)(q - 1)$ i.e. $\gcd(e, (p - 1)(q - 1)) = 1$
3. Compute d such that

$$e \cdot d = 1 \bmod (p - 1)(q - 1)$$

Public key $pk = (N, e)$

Private key $sk = d$

- Encryption: $c = m^e \bmod N$
- Decryption: $m = c^d \bmod N$

Euler Phi function $\phi(N)$

- The totient function of N , denoted with $\phi(N)$, is the order of $\mathbb{Z}_N^*$

Euler's Theorem

For any element x of $\mathbb{Z}_N^*$, we have $x^{\phi(N)} \bmod N = 1$

Why RSA works: correctness

- $ed \bmod (p-1)(q-1) = 1$

- $N = pq$

- $(m^e \bmod N)^d \bmod N$

$$= m^{ed} \bmod N$$

$$= m^{ed \bmod (p-1)(q-1)} \bmod N$$

$$= m \bmod N$$

← Euler's theorem

Why RSA works: security

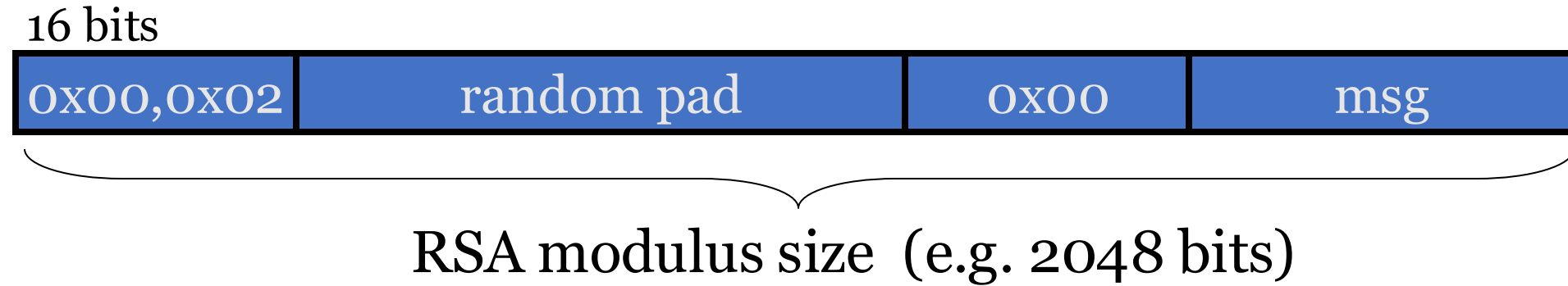
- Factoring assumption: given $N = pq$ where p and q are large prime numbers, it is computationally hard to find p and q
- RSA assumption:
 1. Run $\text{Gen}(1^n)$ to obtain (N, e, d)
 2. Choose a random $y \in \mathbb{Z}_N^*$
 3. A is given (N, e, y) and outputs $x \in \mathbb{Z}_N^*$
 4. Output 1 if $x^e = y \bmod N$

$$\Pr[\text{RSA}_A(n) = 1] \leq \text{negl}(n)$$

Padded RSA

- Gen:
 - $N = pq$, with p and q primes
 - e relatively prime to $\phi(N) = (p - 1)(q - 1)$
 - d inverse of $e \bmod \phi(N)$
 - Public key: $pk = (N, e)$
 - Private key: $sk = d$
- Encryption:
 - Plaintext $m \in \{0,1\}^{\ell_1}$, choose $r \in \{0,1\}^{\ell_2}$ uniformly such that $\ell_1 + \ell_2 = \log N$
 - Interpret $m' = r || m$ as an element in $\mathbb{Z}_N^*$, $c = m'^e \bmod N$
- Decryption:
 - $m' = c^d \bmod N$, output the last ℓ_1 bits of m'

RSA in practice: PKCS#1 v1.5



- Resulting value is RSA encrypted
- Widely deployed, e.g. in HTTPS
- Suffered from a CCA attack

Chosen Ciphertext Attack (CCA) security

Experiment $\text{PubK}_{A,\Pi}^{\text{CCA}}(n)$:

1. Key (pk, sk) is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , pk and $\text{Dec}_{sk}(\cdot)$. A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_{pk}(m_b)$ is given to A .
4. A continues to have **oracle access** $\text{Dec}_{sk}(\cdot)$, but is not allowed to query $\text{Dec}_{sk}(c)$. A outputs b'
5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Definition: a public-key encryption $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is CCA-secure, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{PubK}_{A,\Pi}^{\text{CCA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

Is Plain RSA CCA secure?

- Gen:
 - $N = pq$, with p and q primes
 - e relatively prime to $\phi(N) = (p-1)(q-1)$
 - d inverse of $e \bmod \phi(N)$
- Keys:
 - Public key: $pk = (N, e)$
 - Private key: $sk = d$
- Encryption:
 - Plaintext m in $\mathbb{Z}_N^*$
 - $c = m^e \bmod N$
- Decryption:
 - $m = c^d \bmod N$

$$c = m^e$$

What's the encryption of $2m$?

$$(2m)^e = 2^e \cdot m^e = 2^e \cdot c$$

Attack:

- Challenge: c
- Decryption Query: $2^e \cdot c$
- $2m_0$ or $2m_1$?

Is Padded RSA CCA secure?

- Gen:

- $N = pq$, with p and q primes
- e relatively prime to $\phi(N) = (p - 1)(q - 1)$
- d inverse of $e \bmod \phi(N)$
- Public key: $pk = (N, e)$
- Private key: $sk = d$

- Encryption:

- Plaintext $m \in \{0,1\}^{\ell_1}$, choose $r \in \{0,1\}^{\ell_2}$ uniformly such that $\ell_1 + \ell_2 = \log N$
- Interpret $m' = r || m$ as an element in $\mathbb{Z}_N^*$, $c = m'^e \bmod N$

- Decryption:

- $m' = c^d \bmod N$, output the last ℓ_1 bits of m'

Attack:

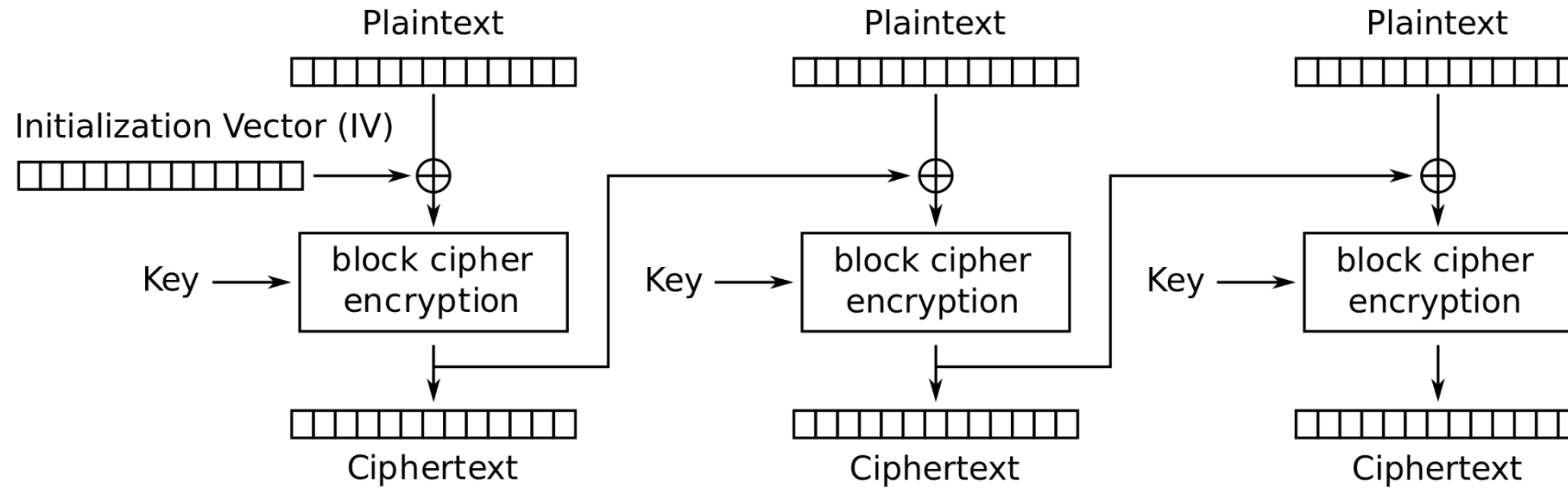
- Challenge: c
- Decryption Query: $2^e \cdot c$
- $2m_0$ or $2m_1$?

Malleability

An encryption algorithm is "malleable" if it is possible to transform a ciphertext into another ciphertext which decrypts to a related plaintext

- Given $c = Enc_k(m)$, it is possible to generate c' such that $m' = Dec_k(c') = f(m)$ for a known function f without knowing m

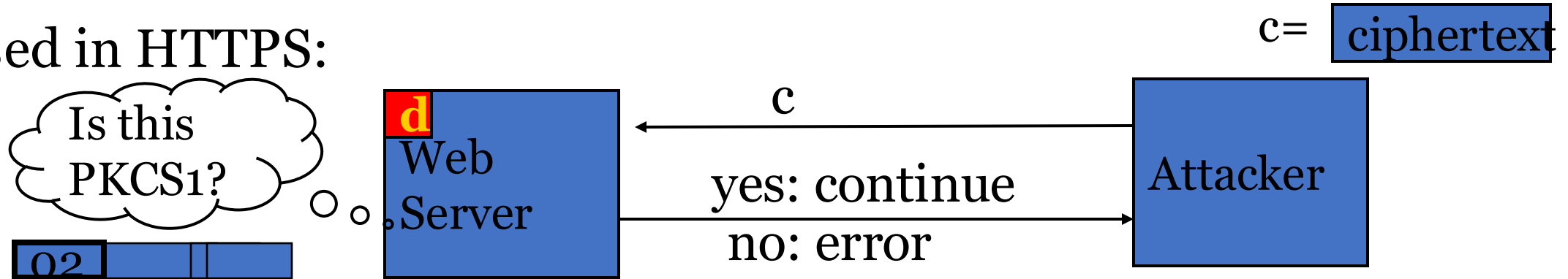
Example: Cipher Block Chaining (CBC) Mode



Cipher Block Chaining (CBC) mode encryption

Attack on PKCS1 v1.5 (Bleichenbacher 1998)

PKCS1 used in HTTPS:

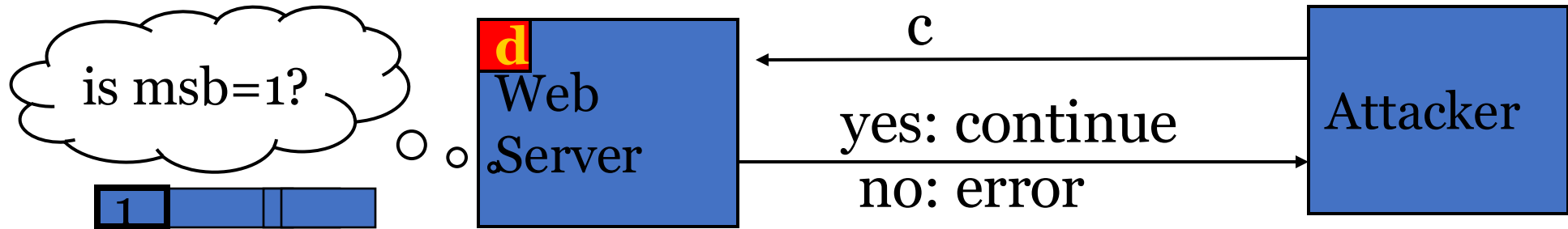


⇒ attacker can test if 16 msbs of plaintext = '02'

Toy example of the attack

compute $m' \leftarrow c^d \bmod N$

$c =$ ciphertext

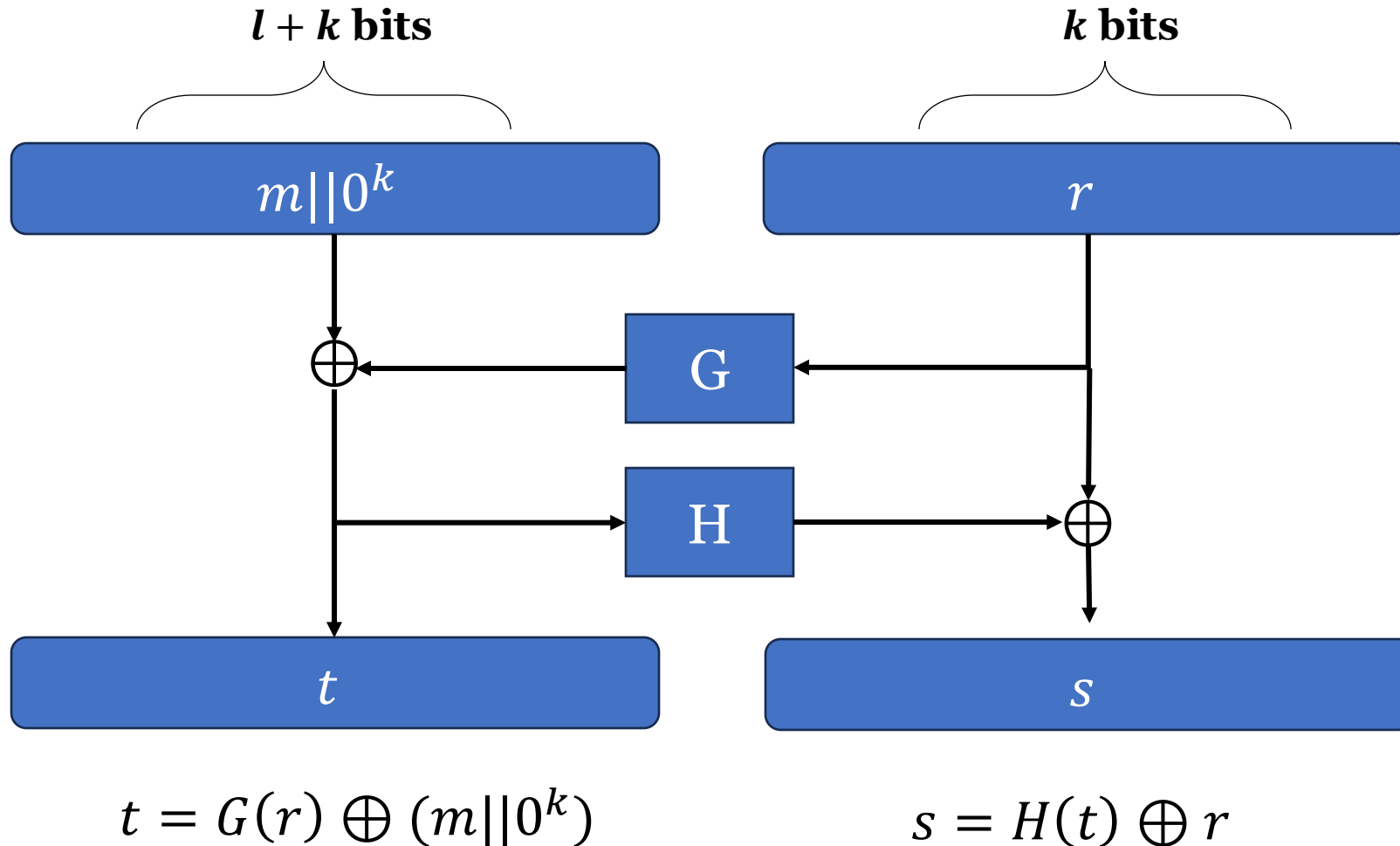


Suppose $N = 2^n$ (an invalid RSA modulus). Then:

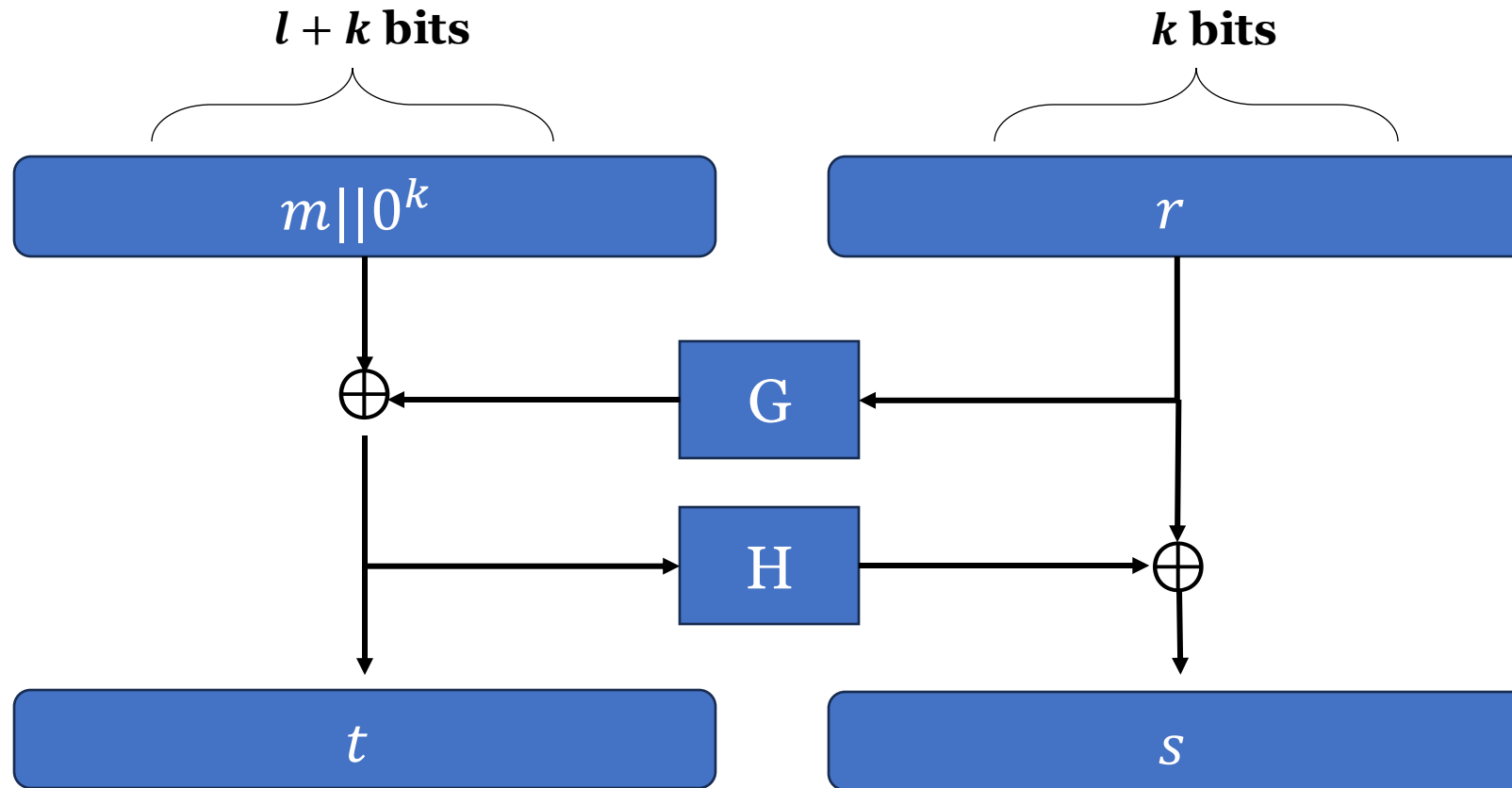
- Sending c reveals $\text{msb}(m')$
- Sending $2^e \cdot c = (2m')^e \bmod N$ reveals $\text{msb}(2m' \bmod N) = \text{msb}_2(m')$
- Sending $4^e \cdot c = (4m')^e \bmod N$ reveals $\text{msb}(4m' \bmod N) = \text{msb}_3(m')$
- ... and so on to reveal all of x

Achieving CCA security: OAEP and PKCS#1 v2

- OAEP: optimal asymmetric encryption padding



OAEP



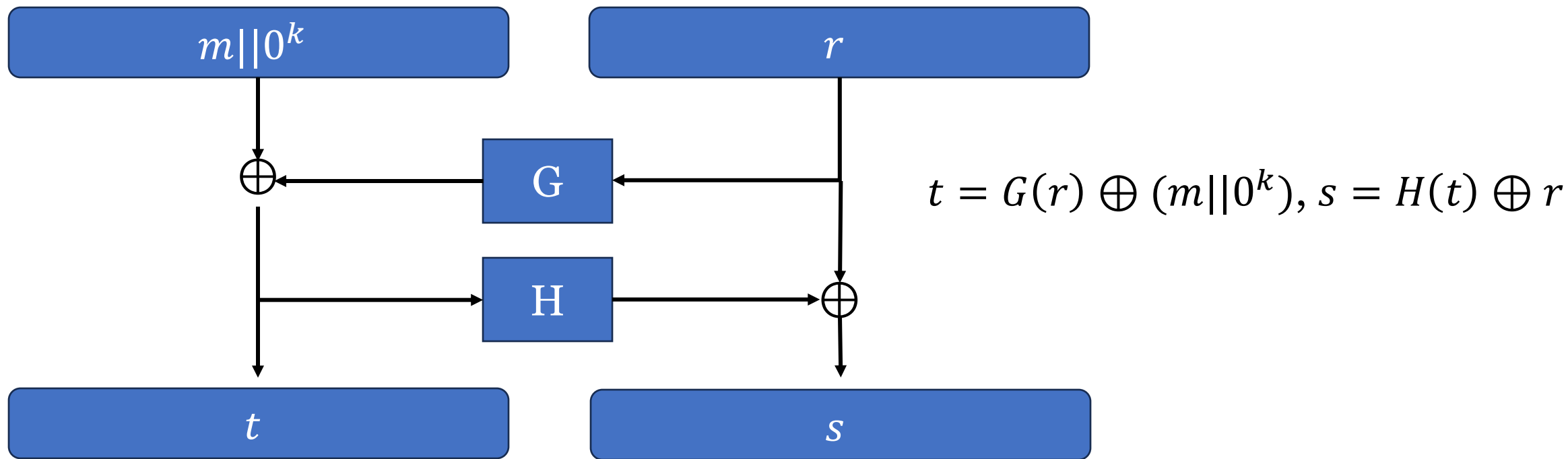
$$G: \{0,1\}^k \rightarrow \{0,1\}^{l+k}, H: \{0,1\}^{l+k} \rightarrow \{0,1\}^k$$

PKCS#1 v2

- Encryption: $c = (t||s)^e \bmod N$
- Decryption: obtain $t||s = c^d \bmod N$
 - $r = s \oplus H(t)$
 - $m^* = t \oplus G(r)$
 - Return error if m^* does not end with k 0s. Otherwise, return the first l bits as the plaintext.

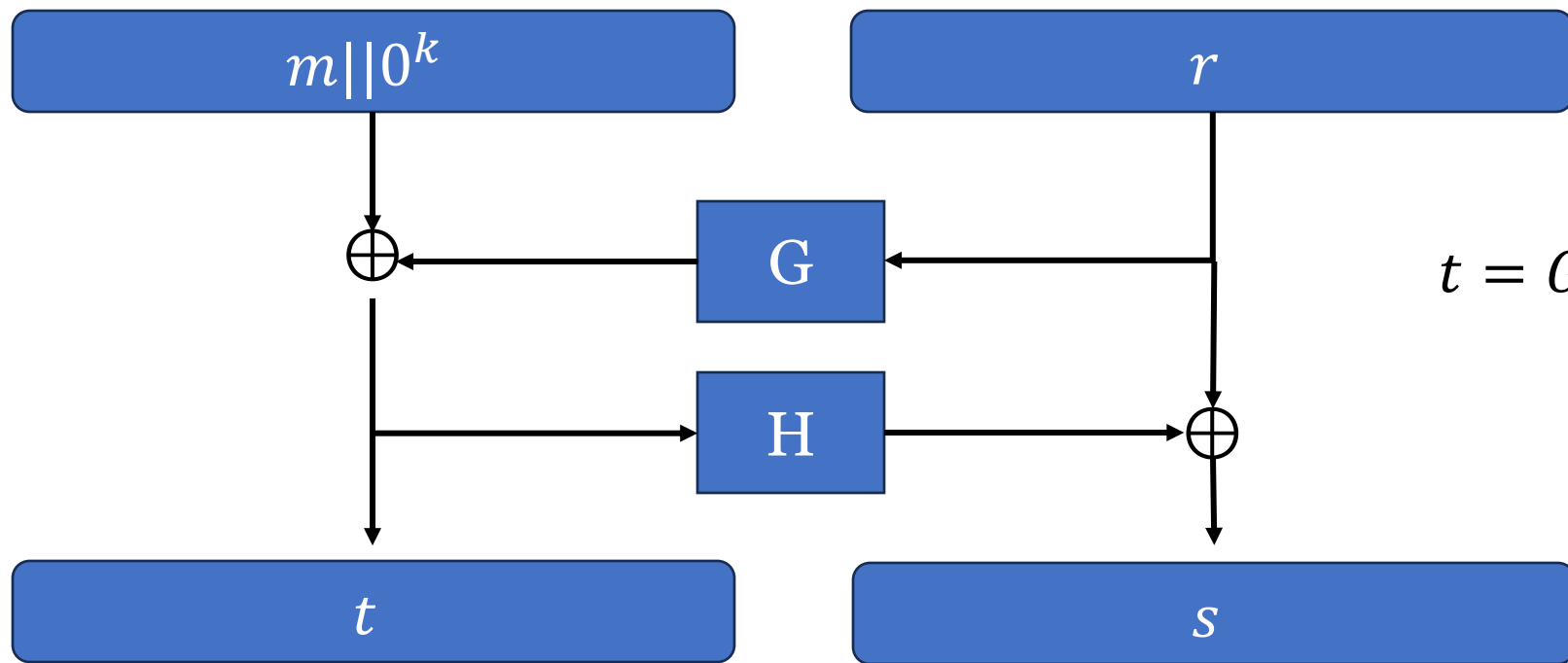
$$t = G(r) \oplus (m||0^k), s = H(t) \oplus r$$

Proof idea: CPA security



1. Does the adversary compute $G(r)$? If not, no information about m
2. Can the adversary compute $G(r)$? Need to compute $H(t)$, except for negl prob.
3. Can the adversary compute $H(t)$? Need to compute t from $(t || s)^e \bmod N$

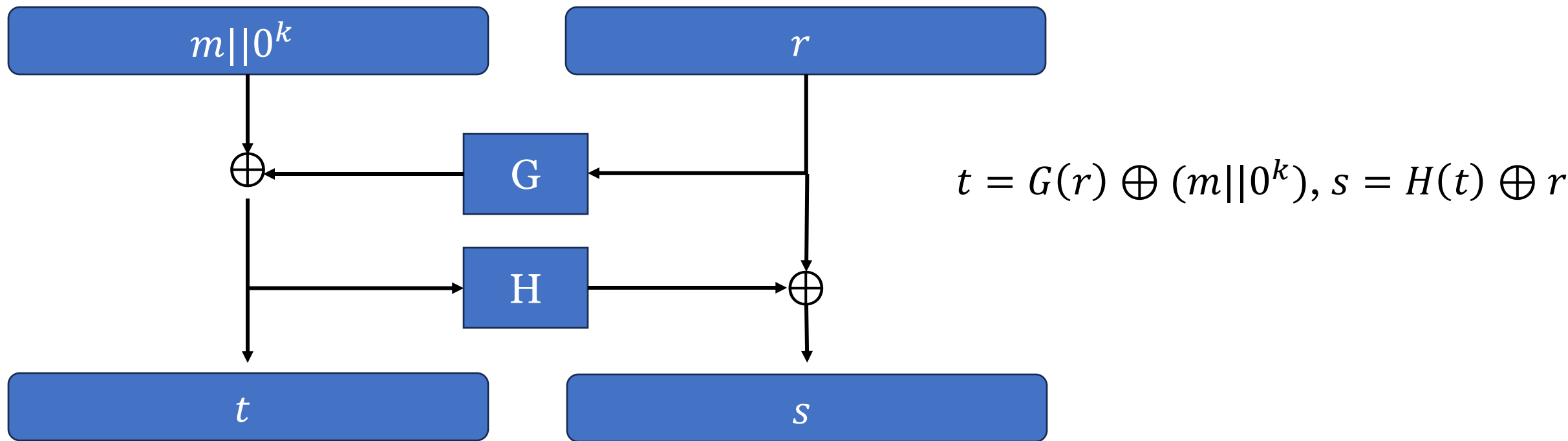
Proof idea: CCA security



$$t = G(r) \oplus (m||0^k), s = H(t) \oplus r$$

Not malleable anymore! Message of $2^e \cdot (t||s)^e$?

Proof idea: CCA security

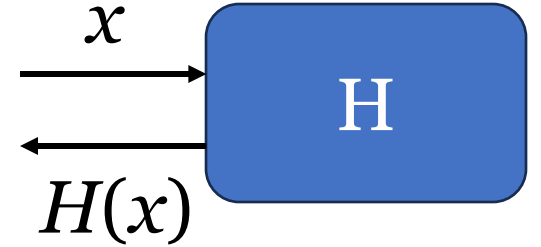


Decryption oracle: can't have 0^k after m , unless ciphertext from encryption oracle

Random Oracle

An oracle that behaves like a random function

- If x has not been queried, $H(x)$ is uniform
- If x has been queried, return the same $H(x)$
- Record every query x to the oracle



Random Oracle: “ideal” hash function

- Public random function

000000	random string 1
000001	random string 2
000010	random string 3
...	...

Collision resistance from RO

- Find $x \neq x'$ such that $H(x) = H(x')$

Birthday attack: probability is $O(\frac{q^2}{2^l})$

PRG from RO

- $G(x) = H(x||0)||H(x||1)$

PRF from RO

- $F_k(x) = H(k||x)$

Can't prove security using collision resistant only

There are counter examples where this is not secure with collision resistant hash function.

Advantage of RO

- Many schemes can only be proven secure in the RO model.
 - OAEP, digital signatures
- There are variants in the standard model without RO, but less efficient.

Problem of random oracle

- Size of a random function is exponential, time for evaluation is also exponential*

A hash function can't be a random oracle.

<https://en.wikipedia.org/wiki/SHA-2>

Problem of random oracle

Can we assume that a hash function H is an RO for PPT adversaries?

Attempt: for all PPT distinguisher D

$$|\Pr[D^{H(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

For all PPT distinguisher D , there is a negligible function negl such that

$$|\Pr[D^{H(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

A PRF is a function $F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$ such that for any PPT distinguishers D , there is a negligible function negl such that

$$|\Pr[D^{F_k(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

For uniformly random $k \in \{0,1\}^n$ and a random function $f \in \text{Func}_n$

Problem of random oracle

- Can't assume that a hash function H is an RO for PPT adversaries

Attempt: for all PPT distinguisher D

$$|\Pr[D^{H(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

H is public!

Problem of random oracle

- Can't assume that a hash function H is an RO for PPT adversaries

$D^{H(\cdot)}$ is the wrong notation for a public hash function H !

Hash with a key/family of hash function doesn't help!